Enhance UAV Network Resilience by Malicious Traffic Detection: A Twin Graph Encoder Approach

Xuzeng Li, *Graduate Student Member, IEEE*, Tao Zhang, *Member, IEEE*, Jiacheng Wang, *Member, IEEE*, Jiangtian Nie, Jian Wang, Xuangou Wu, *Member, IEEE*, Zhen Han, Jiqiang Liu, *Senior Member, IEEE*, Dusit Niyato, *Fellow, IEEE*, and Dong In Kim, *Life Fellow, IEEE*

***Abstract*—Uncrewed aerial vehicle (UAV) networks are increasingly exposed to widespread and various network attacks due to their fully distributed nature and the limited defensive capabilities of individual devices. Existing defense strategies rely on network connectivity and UAV status information, which overlook information of network traffic. Malicious traffic detection offers a promising solution to achieve fine-grained attack detection. However, the dynamic nature and complexity of UAV networks limit the effectiveness of traditional traffic detection methods. Current approaches either fail to fully exploit the raw characteristics of traffic or do not consider the timeliness requirements of UAV networks. To address these challenges, we propose a novel twin graph encoder neural network, which can extract features of raw traffic bytes for efficient traffic detection. First, we propose a decoupled architecture for model training and inference to enable efficient detection of malicious traffic in UAV networks. Second, we propose a novel modeling method that models traffic as the co-occurrence graph and word frequency graph based on raw bytes. Then, we propose TGE-ETD, a Twin Graph Encoder for Encrypted Traffic Detection. TGE-ETD consists of a set of twin graph encoders that effectively extract intrinsic traffic features from graphs constructed from raw bytes. In addition, TGE-ETD employs a global attention pooling mechanism to effectively distinguish the feature contributions of different bytes. Finally, we conducted extensive experiments on a real UAV traffic dataset and four real-world network traffic datasets. TGE-ETD achieved an improvement of 1%-20% over the baseline methods by reducing the number of parameters by 20 times. Tested on multiple UAV hardware devices, TGE-ETD can achieve millisecond-level traffic detection.**

***Index Terms*—UAV networks, malicious traffic detection, graph representation, graph learning.**

I. Introduction

With the rapid development of the low-altitude economy, uncrewed aerial vehicle (UAV) networks have been widely deployed in military and civilian domains [1], including applications such as battlefield surveillance and disaster response [2], [3], [4], [5]. UAVs often operate in swarms to perform data collection or model inference tasks, collaborating with ground-based or cloud servers to form heterogeneous UAV networks [6]. However, the fully distributed nature and structural heterogeneity of such networks expose them to significant security threats. In addition, the limited computational resources of UAV devices restrict their defensive capabilities, thus broadening the attack surface [7], [8], [9], [10], [11], [12]. These challenges highlight the urgent need for robust and broadly applicable defense strategies tailored for heterogeneous UAV networks.

Secure communication is a fundamental prerequisite to ensure stable, reliable and safe data transmission and flight control in UAV networks [9]. Attacks targeting the communication infrastructure of UAV networks can severely degrade their quality of service and overall performance [13]. Existing research on UAV network security can be broadly categorized into attack [7], [8], [13] and defense methods [14], [15], [16], [17], [18]. Attack methods primarily aim to analyze the implementation strategies and potential impacts of attacks on UAV systems. Defense methods focus on designing detection techniques that are tailored to specific attacks. However, many of these approaches either rely on connectivity or behavioral features of UAV networks, overlooking valuable information embedded in communication data, and thus fail to enable diversified and fine-grained attack detection.

Received 26 October 2025; revised 13 February 2026 and 31 March 2026; accepted 3 May 2026. Date of publication 13 May 2026; date of current version 29 May 2026. This work was supported in part by Beijing Natural Science Foundation under Grant L251041; in part by the National Natural Science Foundation of China (NSFC) under Grant 62402029; in part by the Open Fund of State Key Laboratory of Internet Architecture under Grant HLW2025MS02; in part by the Open Fund of Anhui Provincial Key Laboratory of Digital Twin Technology in Metallurgical Industry under Grant ADW24-01; in part by the Open Fund of State Key Laboratory of Integrated Services Networks under Grant ISN27-22. The associate editor coordinating the review of this article and approving it for publication was M. J. Khabbaz. *(Corresponding author: Tao Zhang.)*

Xuzeng Li and Tao Zhang are with the School of Cyberspace Science and Technology, Beijing Jiaotong University, Beijing 100044, China, and also with the State Key Laboratory of Internet Architecture, Tsinghua University, Beijing 100084, China (e-mail: 22110130@bjtu.edu.cn; taozh@bjtu.edu.cn).

Jiacheng Wang and Dusit Niyato are with the School of Computer Science and Engineering, Nanyang Technological University, Singapore 639798 (e-mail: jiacheng.wang@ntu.edu.sg; dniyato@ntu.edu.sg).

Jiangtian Nie is with the School of Natural and Computing Sciences, University of Aberdeen, AB24 3FX Aberdeen, U.K. (e-mail: jiangtian.nie@abdn.ac.uk).

Jian Wang, Zhen Han, and Jiqiang Liu are with Beijing Key Laboratory of Security and Privacy in Intelligent Transportation and the School of Cyberspace Science and Technology, Beijing Jiaotong University, Beijing 100044, China (e-mail: wangjian@bjtu.edu.cn; zhan@bjtu.edu.cn; jqliu@bjtu.edu.cn).

Xuangou Wu is with the School of Computer Science and Technology, Anhui University of Technology, Anhui 243002, China (e-mail: wuxgou@ahut.edu.cn).

Dong In Kim is with the Department of Electrical and Computer Engineering, Sungkyunkwan University, Suwon 16419, South Korea (e-mail: dongin@skku.edu).

Digital Object Identifier 10.1109/TCOMM.2026.3693173

Malicious encrypted traffic detection has emerged as a critical technology in network management, playing a pivotal role in the resolution of attack challenges [19]. It is particularly valuable for improving the resilience and security of UAV networks. Existing approaches to encrypted traffic detection can be broadly classified into knowledge-based filtering methods [20], [21] and learning-based detection methods [22], [23], [24], [25], [26], [27], [28], [29], [30]. Knowledge-based methods, such as port-based identification and deep packet inspection (DPI), rely on predefined rules to classify traffic. However, as network environments become increasingly complex, with the widespread adoption of non-standard interfaces and dynamic port allocation, the effectiveness of these methods has decreased significantly [30]. Furthermore, encryption transforms payloads into ciphertext, rendering DPI ineffective in the general applicability of knowledge-based techniques [27]. To overcome these limitations, researchers have proposed learning-based encrypted traffic detection methods, leveraging the powerful feature extraction capabilities of deep neural networks. Although these approaches have shown impressive performance, they often suffer from two key drawbacks, including inadequate utilization of raw encrypted traffic features and lack of consideration of the resource constraints inherent in UAV networks. As a result, the complexity of these models hampers their practical deployment in UAV scenarios. Moreover, the highly dynamic nature of UAV networks and the heterogeneity of devices increase the complexity of encrypted traffic. As a result, traditional feature engineering methods become ineffective as they cannot extract explicit information from encrypted packets, such as five-tuple features. Meanwhile, existing deep learning–based approaches lack systematic modeling of UAV network traffic characteristics, hindering their ability to capture the latent features of UAV traffic, thus constraining their detection performance.

To address these challenges, we propose a resilience enhancement strategy for UAV networks based on malicious encrypted traffic detection [31], which leverages encrypted traffic information within the network to enable accurate attack detection. Specifically, we design a malicious encrypted traffic detection architecture tailored for UAV networks, which decouples the training and inference of detection models between UAVs and servers, thereby mitigating the impact of limited computational resources. We further introduce TGE-ETD, a Twin Graph Encoder for Encrypted Traffic Detection. TGE-ETD incorporates a hybrid modeling technique that fuses co-occurrence graphs and term frequency graphs, along with a lightweight yet expressive twin graph encoder built upon graph neural networks (GNNs), to effectively extract discriminative features from encrypted UAV network traffic [32]. To validate the effectiveness of our method, we conducted extensive comparative and ablation experiments on five widely used public datasets. Experimental results demonstrate that TGE-ETD consistently performs better than benchmarks. The main contributions are summarized as follows.

1) We propose a novel architecture to improve the resilience of UAV networks that decouples the training and inference phases of malicious encrypted traffic detection models. By offloading model training to cloud servers and enabling collaborative inference with UAVs, the architecture addresses the computational limitations of UAVs while ensuring efficient attack detection.
2) We propose TGE-ETD, a malicious traffic detection approach. It employs a modeling method that integrates co-occurrence graphs and word frequency graphs to represent traffic at the byte level. Furthermore, we design a twin graph encoder to extract byte-level features and incorporate a global attention pooling mechanism to differentiate the contribution of each node in the traffic graph enabling accurate attack detection.
3) We conducted comprehensive experiments based on five real-world datasets, including comparative and ablation studies. The results show that TGE-ETD reduces the number of model parameters by a factor of 20 compared to the baseline methods, while consistently achieving a 1%-20% improvement in performance.

II. Related Works

A. Attacks Detection in UAV Networks

UAV networks are exposed to a wide spectrum of security threats [33], including but not limited to denial of service attacks, jamming attacks, and spoofing attacks [10], [34]. Due to their distributed nature and complex network topology, UAV networks inherently present multiple attack surfaces. At the device level, UAVs are vulnerable to threats such as malware infections, sensor spoofing, and side-channel attacks [9]. During communication, UAV networks face various attacks, including flooding and sybil attacks [9]. At the network architecture level, threats such as man-in-the-middle attacks and data falsification attacks are also prevalent [9]. Research on deep learning-based attack detection in UAV networks is still in its early stages. Existing studies rely on UAV log data [17], [18], [35] or flight data [36], [37] for attack and anomaly detection. Although these approaches can partially mitigate security threats, they depend heavily on expert knowledge, or it is hard to achieve a precise identification of attacks.

B. Malicious Traffic Detection

1) Feature Engineering Methods: As traditional methods are no longer applicable to complex encrypted traffic scenarios [38], [39], machine learning has begun to be widely used to classify encrypted traffic through feature engineering. Taylor et al. [40], [41] designed extracts traffic features based on flow length and packet size information to distinguish traffic from different mobile applications. Liu et al. [42] used the Markov model to extract features from the packet length sequence and the message type sequence. Shen et al. [43] used the length of packets in bidirectional client-server interactions as features and extracted sequence features or statistical features for traffic classification. However, feature engineering methods rely on manually constructed features to identify types of traffic, such as maximum packet length, average delay, and so on. Universality of these methods is difficult to guaranty because manually constructing features costs a lot and they rely on expert experience.

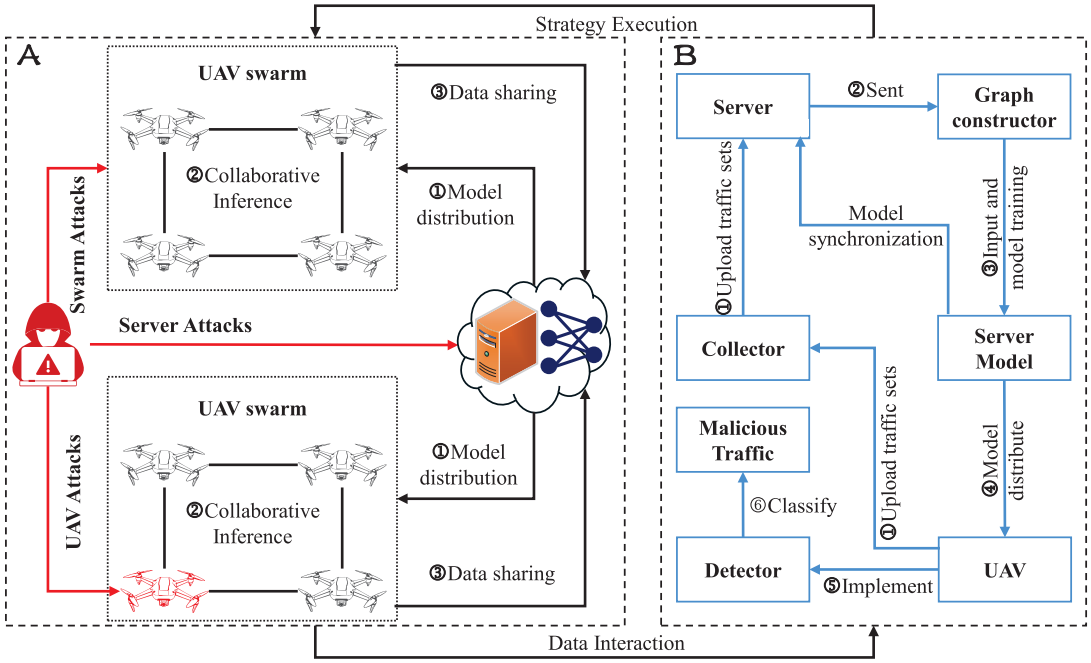


Fig. 1. Proposed resilient UAV network architecture based on malicious encrypted traffic detection. The part A of the figure illustrates the network model, highlighting the various attacks during UAV collaborative inference. The part B in the right presents the execution workflow of the architecture.

2) Deep Learning Methods: As deep learning shows powerful feature extraction capabilities, it has been applied to traffic classification. Lotfollahi et al. [44] proposed Deep-Packet, which uses stacked autoencoders and CNN to extract the features of the traffic packet. Liu et al. [25] and Wang et al. [45] used a recurrent neural network to extract features of the packet length sequence. Meng et al. [46] used variational autoencoders to detect unknown malicious traffic. Zhang et al. [47] designed a time-cyclic attention mechanism to identify the attack patterns of malicious traffic. Shen et al. [48] and Ding et al. [49] used contrastive learning and adversarial training to detect malicious traffic, which can effectively defend against evasion attacks. Zhang et al. [47] realized lightweight malicious traffic detection using an innovative model-splitting algorithm and a hardware specific unsupervised decision tree. Xu et al. [50] took CNN and long short term memory network as teacher models to extract the structural features of traffic to achieve lightweight malicious traffic detection. Cai et al. [51] used cycle generative adversarial networks to capture traffic features. Although these methods have achieved high accuracy, they cannot fully capture the original traffic information.

In order to extract traffic features more effectively, graph learning is widely used because of its powerful representation ability. Shen et al. [29] constructed a traffic interaction graph to extract features from traffic bursts. Zhang et al. [30] proposed the idea of representing the original bytes of encrypted traffic as a graph and designed a graph neural network for encrypted traffic classification. Han et al. [52] combined the traffic interaction graph and the original byte graph to extract traffic features using GNNs. However, GNNs-based methods rely on a single modeling approach and fail to prioritize the detection of malicious encrypted traffic.

3) Pretraining Methods: In order to make full use of the original information of traffic, pre-trained models have been widely used. He et al. [28] proposed PERT based on dynamic word embedding, which takes the original bytes of the traffic packet as input and extracts the original byte features using the dynamic word embedding method. Lin et al. [26] proposed ET-BERT based on BERT [53], which converts the flow into word-like tags and extracts characteristics from unlabeled traffic in a pre-trained manner. Zhao et al. [27] converted the packet bytes into a multi-level flow representation matrix and then pre-trained the model to extract features from unlabeled traffic. Yang et al. [54] designed a pre-trained model based on packet length sequence to achieve stable traffic analysis. Although these methods achieved inspired results, their complexity and high computational resource requirements make them difficult to deploy in UAV networks.

III. System Overview

A. Network Model

As illustrated in part A of Fig. 1, the UAV network framework comprises multiple UAV swarms and a cloud server that collaboratively execute tasks [17], [31]. Taking collaborative inference as an example, the server first distributes the inference model to the UAVs, which then perform inference locally. During this process, individual UAVs, UAV swarms, and the cloud server are all potential targets of attackers. Due to their operational independence and reliance on wireless communication, individual UAVs are particularly vulnerable to direct attacks. Attackers may exploit system vulnerabilities to implant malicious software, thus gaining control over the UAV or disrupting its mission [17]. The interdependent nature of UAV swarms makes them especially susceptible to single-point failures, which compromise of a single node may trigger

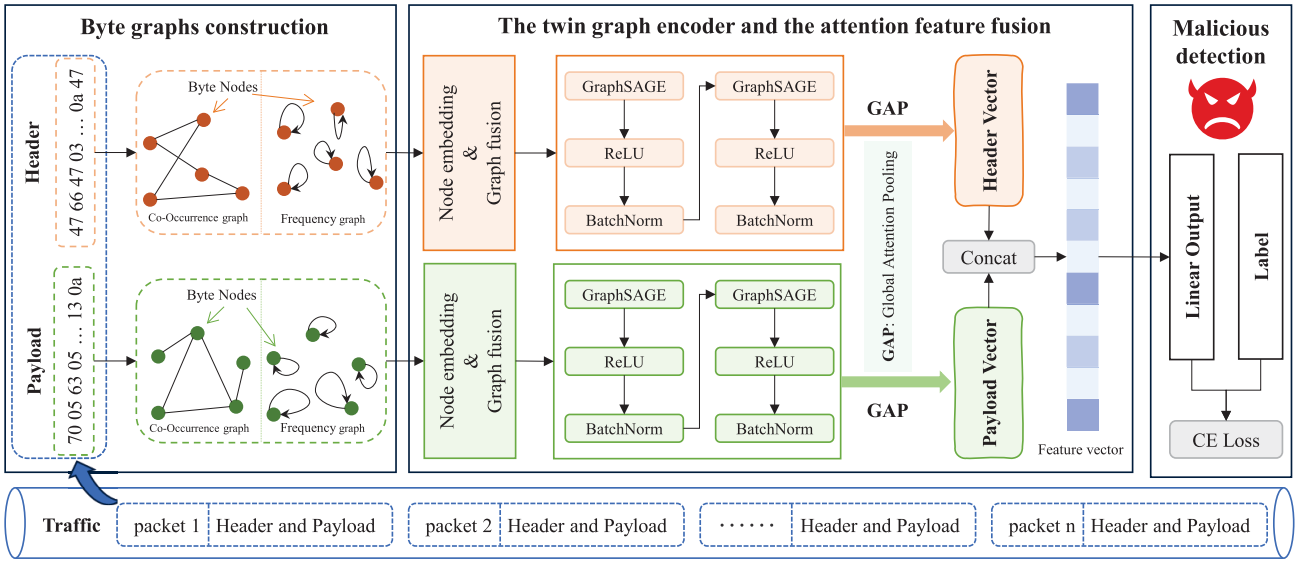


Fig. 2. The architecture of TGE-ETD. (a) Byte graphs construction: Build the co-occurrence graph and frequency graph using the original bytes; (b) The twin graph encoder and the attention feature fusion: Fuse the co-occurrence graph and word frequency graph and perform feature embedding, then extract the original byte features using GNN and obtain the final feature representation based on the global self-attention mechanism; (c) Malicious detection: Obtain the label of encrypted traffic.

cascading failures. In addition, they are exposed to distributed denial-of-service (DDoS) and flooding attacks. The cloud server, which serves as the data processing and storage hub for the UAV system, faces threats such as malware and DDoS attacks, which not only jeopardize data security, but can also disrupt the operation of the entire UAV network.

B. System Model

The attack detection architecture based on malicious encrypted traffic detection in part B of Fig. 1 that illustrates the coordination between the cloud server and the UAVs [17]. The server first collects and processes encrypted traffic from the UAV network. Then, based on the proposed byte-level encrypted traffic modeling approach, the encrypted traffic is transformed into traffic graphs. These graphs are used as input to the designed twin graph neural network for model training. Once trained, the resulting attack detection model is distributed to UAVs for inference. This architecture decouples model training from detection, thus fully leveraging the computational resources of the server while alleviating the burden on resource-constrained UAVs.

C. Analysis of Network Resilience

The accuracy and timeliness of the malicious traffic detection framework will impact on the resilience of the UAV network. Taking collaborative UAVs inference scenarios as an example, accurate detection constrains the propagation of anomalous traffic among UAVs, thus mitigating packet loss and improving the packet delivery ratio. Let $\mathcal{U}_s$ denote a UAV swarm participating in collaborative inference, where inference results are exchanged through encrypted communication. The probability of successfully detecting malicious traffic within a single detection interval is denoted as p_d. We assume the number of rounds N until the first successful detection follows a geometric distribution $Pr(N = \mathbf{k}) = (1 - p_d)^{\mathbf{k}-1} p_d$. If each detection interval lasts for Δt, the expected detection time can be expressed as $\mathbb{E}[T_{\text{det}}] = \Delta t / p_d$. Higher detection accuracy p_d enables UAVs to effectively filter malicious traffic, while improved detection timeliness Δt reduces the duration for which the network remains exposed to attacks. It is evident that prolonged exposure of the network to malicious attacks leads to a lower task success rate, a higher packet loss ratio, and reduced inference reliability.

IV. Methodology

In this section, we first describe the malicious encrypted traffic detection task, and then introduce TGE-ETD in detail. The overview of TGE-ETD is shown in Fig. 2, which mainly includes byte graph construction, twin graph encoder, and attention-based feature fusion.

A. Task Description

The malicious encrypted traffic detection task is to use the information from traffic packets captured by professional software to distinguish malicious traffic (produced by attacks) from normal traffic. In this paper, we perform fine-grained malicious encrypted traffic detection, specifically identifying the specific categories of malicious traffic packets.

B. Formulation

We define a traffic packet as a sample and detect malicious packets, which is common practice in many encrypted traffic methods. A session $\mathbf{S} = [\mathbf{p}_1, \mathbf{p}_2, \ldots, \mathbf{p}_n]$ (bidirectional flow) in the network consists of exchanged request and response packets between two communicating entities in the network, where n is the length of the packet sequence and $\mathbf{p}_i$ is

the i-th packet in the session. The packet is represented as $\mathbf{p}_i = [b_1, b_2, \ldots, b_m]$, where m is the length of the byte sequence and b_j is the j-th byte of i-th packet. According to the definition above, the (packet-level) malicious encrypted traffic detection task is to identify whether an unseen traffic packet $\mathbf{p}_i$ is malicious and identify its specific category C_k.

C. Byte Graphs Construction

In our detection task, we focus on network traffic packets. Previous studies have shown that the header and payload parts of traffic packets contain different information and contribute differently to network detection capabilities, so we divide traffic packets into header and payload. Each packet can be described as $\mathbf{p} = [header, payload]$. We choose to use raw bytes to represent $header$ and $payload$ instead of statistical information because the raw bytes information is more accurate and comprehensive. For feature extraction of raw bytes, we consider three aspects including different bytes having different meanings, different positions of specific bytes in the context contain different information, and the frequency of the same bytes appearing in different parts of the data packet is different. Therefore, we construct a byte graph based on these three aspects of information to represent the traffic packet.

In addition, to distinguish the same bytes at different positions in the byte sequence, we treat the repeated bytes as separate nodes in the graph (the number of nodes does not exceed 256 [27], [30]), which can avoid the problem of the number of graph nodes exploding. In order to effectively pass messages between nodes and fully extract the contextual information, we build two types of graphs for each part, including the co-occurrence graph and the word frequency graph. According to the above description, we convert the original bytes of $header$ and $payload$ into the co-occurrence graph $\mathcal{G}_o = \{\mathcal{V}_o, \mathcal{E}_o, \mathcal{X}_o\}$and the word frequency graph $G_f = \{\mathcal{V}_f, \mathcal{E}_f, \mathcal{X}_f\}$ based on the correlation between the bytes.

1) Co-Occurrence Graph Construction: For the original byte representation s of the $header$ ($payload$) of a traffic packet. To extract the correlation features between different bytes in the packet, we use the co-occurrence matrix to establish the relationship between each byte [32]. We construct the corresponding co-occurrence graph $\mathcal{G}_o = \{\mathcal{V}_o, \mathcal{E}_o, \mathcal{X}_o\}$, where $\mathcal{V}_o$ is the set of nodes, each byte contained in s corresponds to a node, and $\mathcal{X}_o$ is the features of the nodes. We process s using a sliding window of fixed size default to 3 (according to the ablation experiments in Fig. 11). For bytes that appear in the same window, we construct the co-occurrence edge $\mathcal{E}_o$ between these bytes. Specifically, as shown in Fig. 3, for s, we first use the sliding window to construct the byte co-occurrence matrix $\mathcal{A}$, and $\mathcal{A}$ is the adjacency matrix of $\mathcal{G}_o$. For bytes i and j in s, we describe the edges between them through the adjacency matrix as follows:

$$a_{ij} = \begin{cases} 1, & (i,j) \quad in \quad W \\ 0, & \text{otherwise} \end{cases}. \tag{1}$$

The features of the nodes in G_o are initialized to 1 and the co-occurrence graphs are undirected.

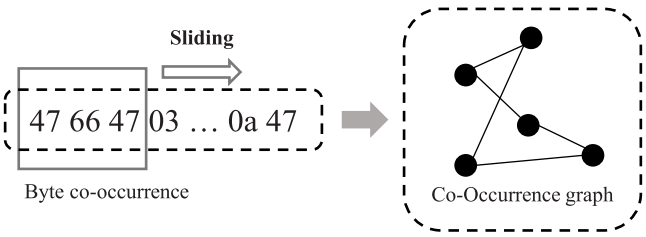


Fig. 3. Construct the co-occurrence graph.

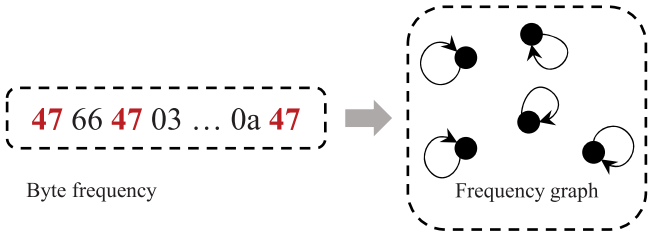


Fig. 4. Construct the word frequency graph.

2) Word Frequency Graph Construction: The frequency of different bytes in the original bytes s of $header$ ($payload$) of the encrypted traffic packet is similar to the frequency of words in NLP tasks, which is an important feature [32]. We construct the word frequency graph $\mathcal{G}_f = \{\mathcal{V}_f, \mathcal{E}_f, \mathcal{X}_f\}$ to extract frequency characteristics. As shown in Fig. 4, each node in $\mathcal{V}_f$ represents a byte in s, the edge set $\mathcal{E}_f$ is the self-loop of each node and $\mathcal{X}_f$ is the node feature. For the feature x_f^i of node i, we use the frequency f_i and the corresponding byte value b_i of i to represent its feature, b_i ranges from 0 to 255. We perform one-hot encoding on b_i in [0, 255] to obtain the vector $B_i^{1\times 256}$, where $B_i[b_i] = 1$ and other elements in B_i are 0. We then multiply the frequency and the original byte value features to get the final representation $x_f^i = f_i \times B_i$.

D. Twin Graph Encoder

The information contained in different parts of the traffic packet is different because the bytes in different positions in the traffic packet reflect different information. The same bytes in $header$ and $payload$ of the traffic packet reflect different information. The original bytes of the encrypted traffic packet $header$ that are not encrypted reflect different information from the original bytes of the encrypted $payload$. The same bytes before and after encryption are different. If we use the same feature embedding representation for the same bytes in $header$ and $payload$, it will generate confusing model inputs, resulting in sub-optimal model performance. We designed a twin graph encoder to process $header$ and $payload$ of the traffic packet and then fuse the embedding vectors of $header$ and $payload$ through the designed attention-based feature fusion module to obtain the final embedding representation.

After obtaining the co-occurrence graph and the word frequency graph of the $header$ ($payload$), we fuse the co-occurrence graph $\mathcal{G}_o$ and the word frequency graph $\mathcal{G}_f$ to obtain a unified graph representation $\mathcal{G}_F = \{\mathcal{V}_F, \mathcal{E}_F, \mathcal{X}_F\}$.

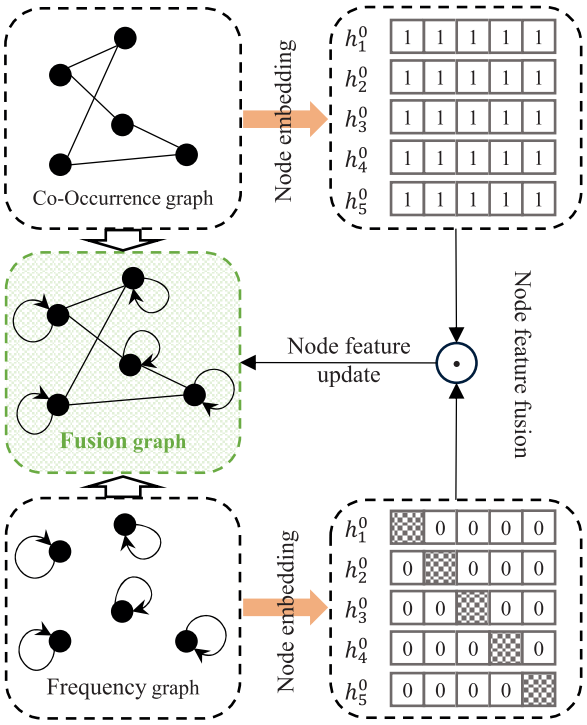


Fig. 5. Fuse co-occurrence and word frequency graph.

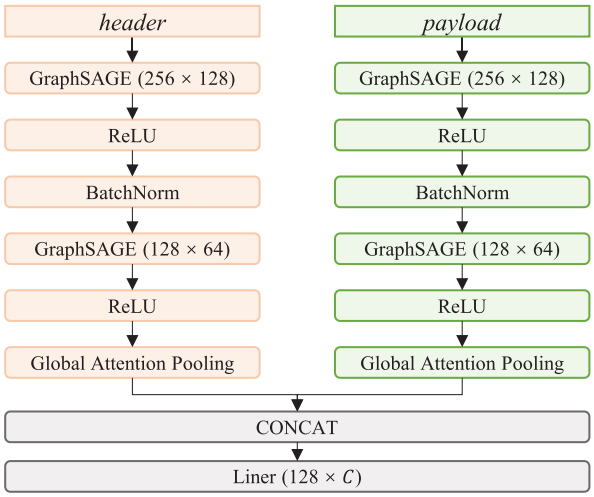


Fig. 6. Details of the network.

Then, we update the embedding representation of the byte node. As shown in Fig. 5, the nodes in the co-occurrence graph and the word frequency graph are all bytes. The graph fusion operation is divided into two parts: edge fusion and node feature fusion. We describe the detailed process of edge fusion in the form of an adjacency matrix. The adjacency matrix $\mathcal{A}_o$ of the co-occurrence graph $\mathcal{G}_o$ and the adjacency matrix $\mathcal{A}_f$ of the word frequency graph $\mathcal{G}_f$ are added to obtain the adjacency matrix $\mathcal{A}_F$ of the fused graph. The node feature fusion processes the node embedding representation in the co-occurrence graph and the word frequency graph, and obtains a new node representation through multiplication operations $\mathcal{X}_F = \mathcal{X}_o \times \mathcal{X}_f$. The node embedding process updates the node representation to the fused node representation.

After processing, both *header* and *payload* are transformed into unified byte-level representations using graphs. Although their original contents differ, the resulting representations are homogeneous byte graphs with identical structural semantics. As a result, the same encoder depth and aggregation strategy are sufficient to capture high-order dependencies in both components. Thus, the twin graph encoder consists of two graph encoders with the same model structure but not shared parameters, which can process the graph structure data of *header* and *payload* in parallel. The structure of the graph encoder is shown in Fig. 6. In order to encode the graph composed of traffic packets into a feature vector, the graph encoder includes two stacked GraphSAGE [55] modules. For each node v on the graph $\mathcal{G}$, GraphSAGE first uses the degree of the target node to normalize the embedding vectors of all neighbor nodes [30]. Then GraphSAGE concatenates the embedding vectors of the target vertex and the neighbor vertex and averages each dimension of the vector. Finally, the generated new target node embedding vector is computed by a nonlinear transformation to complete the forward propagation of GraphSAGE. The information transmission process of GraphSAGE can be formally described as follows:

$$\mathbf{M}_{\mathcal{N}(v)}^{(l)} = \sum_{u \in \mathcal{N}(v)} \frac{x_u^{(l-1)}}{|\mathcal{N}(v)|}$$
$$x_v^{(l)} = \sigma\left(\mathbf{w}^{(l)} \cdot \mathrm{CONCAT}\left(x_v^{(l-1)}, \mathbf{M}_{\mathcal{N}(v)}^{(l)}\right)\right), \quad (2)$$

where $\mathcal{N}(v)$ is the set of neighbor nodes of the target node v, $|\mathcal{N}(v)|$ is the number of neighbor nodes, $\mathbf{w}^{(l)}$ is the parameter on layer l, $\mathrm{CONCAT}(\cdot)$ denotes the concatenation operation and $\sigma(\cdot)$ denotes the activation function. We use the ReLU activation function to process the output of GraphSAGE to increase the nonlinear relationship between the layers of the network. Lastly, we use batch normalization to normalize the updated feature vector $x_v^{(l)}$.

The number of GraphSAGE layers represents the number of neighbor nodes that each node can aggregate. Too many layers will lead to an over-smoothing issue [56]. Experimental verification shows that the number of layers stacked in the GraphSAGE module does not need to be too large, and high accuracy can be achieved when the number of layers is two [55]. Therefore, we stack two layers of GraphSAGE, and use the output of the first GraphSAGE module as the input of the second block.

E. Attention-Based Deep Feature Fusion

Since we constructed the graph representation of the encrypted traffic packet, the malicious packet detection task became a graph classification task. Therefore, after we obtained the new node feature representation through the graph encoder, we designed a feature aggregation method based on the attention mechanism. The global attention aggregation mechanism assigns different attention weights to the relationships between nodes, allowing the model to pay more attention to important nodes and their features, thereby obtaining more

accurate graph features. Thus, we use the global attention mechanism [57] to aggregate the node feature to obtain the representation of the entire graph, and formulate as follows:

$$f_{\mathrm{a}} = \sum_{k=1}^{N} \alpha_i x_k, \quad \alpha_i = \sigma\left(f_{\mathrm{g}}(k_i)\right), \tag{3}$$

where $f_{\mathrm{g}}(\cdot)$ is a neural network that computes attention scores for each feature, we set $f_{\mathrm{g}}(\cdot)$ as a linear layer $f_{\mathrm{g}}(x_i) = w^{\top} x_i + b$, $\sigma(\cdot)$is the sigmoid function, N is the number of nodes in the graph, and x_k is the feature of the k-th node. Then, we input the node features into the attention-pooling function $f_{\mathrm{a}}(\cdot)$ to get the graph representation. Subsequently, we concatenate the head feature $\mathcal{F}_h$ and the payload feature $\mathcal{F}_p$ to obtain the final encrypted traffic packet representation,

$$\mathcal{F} = \mathrm{CONCAT}\left(\mathcal{F}_h, \mathcal{F}_p\right). \tag{4}$$

F. Malicious Detection

After obtaining the final feature representation of the encrypted traffic packet, denoted $\mathcal{F}$, we use a fully connected neural network to process the features and then use softmax to obtain the detection results of malicious encrypted traffic packets. Subsequently, we use the cross-entropy loss function to calculate the model loss, which is used for back-propagation during training.

$$\hat{y} = \mathrm{Softmax}\left(\mathbf{W} \cdot \mathcal{F} + \mathbf{b}\right), \tag{5}$$

$$\mathrm{Loss} = -\sum_{i=1}^{Q} \sum_{j=1}^{\mathcal{C}} y_i \log \hat{y}_i, \tag{6}$$

where $\mathbf{W}$ and $\mathbf{b}$ are the parameters of the full connected layer, $\hat{y}$ are the predicted values, Q is the quantity of samples, and $\mathcal{C}$ is the number of traffic categories.

V. Experiments and Analysis

In this section, we first introduce the basic experimental settings, including the dataset, data preprocessing, the experimental environment, and the evaluation indicators. Then we introduce comparative experiments, including the introduction of comparative methods, experimental parameter settings, and comparative experimental results analysis. Next, we introduce the ablation experiments, including experimental goal settings, ablation experiment scheme settings, and results analysis, to evaluate the effectiveness of TGE-ETD from the dimensions of sensitivity, accuracy, and model complexity.

A. Datasets

To assess the effectiveness of our method, we use multiple public datasets, including two datasets USTC-TFC2016 [58] and CICIoT2022 [59] for malicious encrypted traffic detection. In addition, to verify the effectiveness of TGE-ETD, we perform experiments on two widely used datasets, ISCX-VPN [60] and ISCX-Tor [61]. USTC-TFC2016 [58] contains ten types of traffic collected from malicious programs and ten types of traffic collected from benign programs. CICIoT2022 [59] contains normal traffic and attack traffic collected from IoT devices. ISCX-VPN [60] contains six types of traffic collected in a Virtual Private Network (VPN). ISCX-Tor [61] contains eight types of traffic collected through The Onion Router (Tor). The detailed label is shown in table I and II.

TABLE I

Labeled USTC-TFC2016 and CICIoT2022 Datasets. The Table I llustrates the Types of Traffic We Classify and How Different Categories of Traffic Are Generated

Dataset	Traffic category	Describe
USTC-TFC2016	BitTorrent	Benign traffic collected from the network environment.
	FTP	
	Facetime	
	Gmail	
	MySQL	
	Outlook	
	SMB	
	Skype	
	Weibo	
	WorldOfWarcraft	
	Cridex	Attack traffic collected from the attack software.
	Geodo	
	Htbot	
	Miuref	
	Neris	
	Nsis-ay	
	Shifu	
	Tinba	
	Virut	
	Zeus	
CICIoT2022	Audio	Traffic captured from the IoT devices such as Dlinkcam, GoSound, *etc*.
	Cameras	
	HomeAutomation	
	Flood	Traffic from the flood attack, Hydra and Nmap on the IoT devices.
	Hydra	
	Nmap	

TABLE II

Labeled ISCX-VPN and ISCX-Tor Datasets. The Table I llustrates the Types of Traffic We Classify and Where the Traffic Comes From

Dataset	Traffic category	Describe
ISCX-VPN	Chat	AIM, Facebook, Hangouts, ICQ and Skype.
	Email	Email.
	File	FTPS, SFTP and Skype.
	P2P	Bittorrent.
	Streaming	Netflix, Spotify, Vimeo and Youtube.
	VoIP	Facebook, Hangouts, Skype and Voipbuster.
ISCX-Tor	Audio	Spotifygateway and Spotify.
	Browsing	Firefox and Chrome.
	Chat	AIM, Facebook, Hangouts, ICQ and Skype.
	Email	Email (POP, IMAP).
	File	FTPS, SFTP and Skype.
	P2P	MultipleSpeed and Vuze.
	Streaming	Vimeo and Youtube.
	VoIP	Facebook, Hangouts and Skype.

B. Data Pre-Processing

The dataset provides the original encrypted traffic in the form of pcap files. Since there is invalid and redundant traffic (flows with a load of 0 and flows with too many packets) in the original traffic, we need to pre-process the original PCAP files. The pre-processing operation includes the following steps.

TABLE III
HYPERPARAMETER VALUES OF TGE-ETD

HyperParameter	Value
The dropout ratio of each convolutional layer	0.2
The dimension of hidden layer features	64
The shape of Linear in Global Attention Pooling	(64, 1)
The number of packets per flow used in the malicious traffic identification dataset	10
The number of packets per flow used in the VPN traffic identification dataset	50
The number of packets per flow used in the Tor traffic identification dataset	600
The epoch number	100

1) Segmentation of the Original PCAP File: To delete invalid traffic, we first use the traffic segmentation tool (Splitcap) to divide the pcap file into bi-flows based on the traffic quintuple information. After the segmentation has been completed, the invalid and redundant traffic is deleted.

2) Data Packet Selection: Our task is to identify malicious encrypted traffic packets, so after obtaining the encrypted traffic flow, we need to select the data packets for model training. However, CICIoT2022 and USTC-TFC contain a large number of bi-flows, on the amount of hundreds of thousands. Thus, we performed sampling on the CICIoT2022 and USTC-TFC2016 datasets, with 1000 bi-flows in CICIoT2022 and 500 bi-flows in USTC-TFC randomly sampled from each class for the experiments. Then, we select a certain number of packets in each bi-flow to reduce redundancy.

3) Data Labeling: We assign labels to each sample according to the official instructions and the existing method [27]. In addition, we randomly map the source and destination IP addresses and ports in the traffic packets, retaining only the relationship information without using the actual numerical value, in order to protect sensitive information and eliminate the introduced noise. It should be noted that our algorithm can process both TCP and UDP packets, but the comparison algorithm cannot process UDP traffic, so we removed Tinba traffic in USTC-TFC2016, as Tinba only contains UDP traffic. Regarding the generalization of real-world UAV traffic, where UDP is widely used, the applicability of baseline methods that can only handle TCP traffic will be significantly reduced. However, since our method inherently supports UDP traffic, its applicability to realistic UAV networks is not constrained by this experimental setting.

C. Experimental Implementation Details

1) Experimental Setup: Table III shows the hyperparameter settings of TGE-ETD in the experiment. We use Adam optimizer to minimize the loss, the initial value of the learning rate is set to 0.005, the mini-batch size is 128, and the learning decay rate is the default 0. The dataset is divided into training and test sets in a ratio of 9:1. Pytorch is used to implement our model and perform experiments on a server with NVIDIA RTX 3090 GPUs and AMD 3970X CPU.

2) Evaluation Metrics: We refer to existing encrypted traffic classification research [27], [30], [52] and take accuracy (ACC), precision (PR), recall (RC), and F_1-score as evaluation metrics. We first calculate the values of True Positive, True Negative, False Positive and False Negative. Then, based on the above definitions, we can calculate the evaluation metrics.

D. Comparison Methods

We select a packet length-based recurrent neural network model FS-Net [25], two pre-trained models including ET-BERT [26] and YaTC [27], and two graph neural network models including GraphDAPP [29] and TFE-GNN [30] as baseline methods.

(1) FS-Net [25]: FS-Net is an end-to-end encrypted traffic classification model based on recurrent neural networks. It takes packet length sequences as input, uses bi-GRU to extract encrypted traffic features, introduces an autoencoder-decoder structure, and designs a reconstruction mechanism to enhance feature effectiveness while extracting sequence features.

(2) ET-BERT [26]: ET-BERT converts the encrypted traffic classification task into a natural language processing task. It takes the raw bytes of encrypted traffic as input, extracts features from unlabeled encrypted traffic through pre-training.

(3) YaTC [27]: YaTC constructs the raw packet bytes in the encrypted traffic flow into a multi-level flow representation matrix, then uses a masked autoencoder to pre-train a classifier, extracts features from unlabeled encrypted traffic, and then fine-tunes the model using a small amount of labeled data.

(4) GraphDAPP [29]: GraphDAPP proposes a traffic interaction graph model. It models the flow as a graph based on the traffic interaction information and packet length information. The nodes are traffic packets, the node features are the packet length and direction, and the interaction relationship between packets is the edge. Then, the graph neural network is used to extract the encrypted traffic features and classify them.

(5) TFE-GNN [30]: TFE-GNN divides the encrypted traffic packet into two parts: header and payload. Each part is constructed as a graph based on the original bytes of the traffic packet. The nodes in the graph are the original bytes, the node features are the vector embeddings of the original bytes, and the association relationship between bytes is the edge. Finally, the graph neural network is used to extract the features of each part, and then the final feature representation of the encrypted traffic is obtained using the designed feature fusion method.

E. Comparison Experiments

The comparative experimental results are shown in Table IV. From the results, we can get the following information.

1) Overall Performance: Compared to the state-of-the-art method, TGE-ETD has improved 0.5%-5% in four indicators. Compared with baseline methods, TGE-ETD improves 5%-15% in four indicators. TGE-ETD has a more obvious improvement in the malicious encrypted traffic detection task. The four indicators on the four datasets are higher than 99%, which reflects the effectiveness of TGE-ETD.

2) Analysis of Deep Learning Methods: The fluctuation of various indicators of deep learning methods based on the packet length in four datasets is greater than 10%. These methods have difficulty accurately identifying various types

TABLE IV
THE RESULTS OF TGE-ETD AND BASELINE METHODS

dataset	ISCX-VPN				ISCX-Tor				CICIoT2022				USTC-TFC2016			
model	ACC	PR	F_1	RC	ACC	PR	F_1	RC	ACC	PR	F_1	RC	ACC	PR	F_1	RC
FS-Net	0.8298	0.8324	0.8245	0.8234	0.2798	0.2790	0.2097	0.2627	0.9534	0.9547	0.9519	0.9537	0.7964	0.8065	0.7747	0.7780
GraphDAPP	0.2188	0.2759	0.2071	0.2188	0.4286	0.2557	0.2281	0.2509	0.1875	0.4099	0.2275	0.1875	0.1047	0.0512	0.0684	0.1047
ET-BERT	0.9917	0.9917	0.9917	0.9917	0.9881	0.9882	0.9881	0.9881	0.9725	0.9736	0.9725	0.9725	0.8947	0.8938	0.8929	0.8947
TFE-GNN	0.9404	0.9303	0.9306	0.9327	0.8457	0.8531	0.8313	0.8535	0.9881	0.9904	0.9893	0.9885	0.9879	0.9699	0.9559	0.9506
YaTC	0.9947	0.9949	0.9947	0.9947	0.9375	0.9132	0.9206	0.9375	0.9941	0.9943	0.9941	0.9941	0.9863	0.9867	0.9864	0.9863
TGE-ETD	**0.9968**	**0.9968**	**0.9968**	**0.9968**	**0.9975**	**0.9975**	**0.9975**	**0.9975**	**0.9954**	**0.9954**	**0.9954**	**0.9954**	**0.9938**	**0.9939**	**0.9937**	**0.9938**

TABLE V
THE RESULTS OF THE ABLATION EXPERIMENTS

dataset	ISCX-VPN				ISCX-Tor				CICIoT2022				USTC-TFC2016			
model	ACC	PR	F_1	RC	ACC	PR	F_1	RC	ACC	PR	F_1	RC	ACC	PR	F_1	RC
w/o-header	0.6454	0.6476	0.6455	0.6141	0.6659	0.6671	0.6603	0.6659	0.8498	0.8551	0.8478	0.8496	0.9401	0.9427	0.9397	0.9401
w/o-payload	0.9924	0.9924	0.9924	0.9924	0.9794	0.9794	0.9793	0.9794	0.9870	0.9870	0.9870	0.9870	0.9774	0.9778	0.9771	0.9774
w/o-oc	0.8506	0.8507	0.8479	0.8503	0.4973	0.4881	0.4673	0.4973	0.8851	0.8854	0.8847	0.8851	0.9474	0.9507	0.9467	0.9474
w/o-fg	0.2346	0.1288	0.1324	0.2346	0.1907	0.1104	0.0986	0.1907	0.3823	0.2655	0.2202	0.3823	0.2049	0.0890	0.0906	0.2049
TGE-ETD-mean	0.9912	0.9913	0.9911	0.9912	0.9972	0.9972	0.9972	0.9972	0.9900	0.9900	0.9900	0.9900	0.9884	0.9886	0.9883	0.9884
TGE-ETD-att	**0.9968**	**0.9968**	**0.9968**	**0.9968**	**0.9975**	**0.9975**	**0.9975**	**0.9975**	**0.9954**	**0.9954**	**0.9954**	**0.9954**	**0.9938**	**0.9939**	**0.9937**	**0.9938**

of traffic because they only consider packet length as a characteristic of encrypted traffic. It also shows the importance of original byte information as a feature of encrypted traffic.

3) Analysis of Pre-Training Methods: The indicators of the pre-training method in the four datasets are all higher than 89%, but there is a fluctuation of 5%-10%. ET-BERT and YaTC achieve high ACC and F_1, because they take the original bytes of encrypted traffic as input and can extract more origin features. However, there is a gap between the method based on pre-training and TGE-ETD. This is because the method based on pre-training does not distinguish the contribution of the original bytes of encrypted traffic and cannot fully extract the original features. YaTC selects fixed header and payload bytes for arrangement, so it achieves relatively better results.

4) Analysis of Graph Learning Methods: GraphDAPP only uses the length and direction as node features, so it cannot effectively extract encrypted traffic features. The TFE-GNN method constructs the original bytes into a graph and divides the original bytes to distinguish the contributions of different parts, achieving high values across all metrics. However, because of the single graph construction method, it is unable to fully extract features. In addition, since TFE-GNN combines multiple encrypted traffic packets to form samples, although it improves the recognition efficiency, the combination operation introduces additional noise, which affects the model effect.

5) Comparative Analysis Between Different Datasets: The traffic in ISCX-Tor is processed by Tor, and encryption and obfuscation technology make it difficult to analyze the payload of anonymous traffic, which affects the effect of the baseline model. However, TGE-ETD directly processes the encrypted traffic packets and uses multiple graph representation methods to model the original bytes, improving the accuracy of feature extraction and improving the model effect. Compared to CICIoT2022, USTC-TFC2016 contains more attacks, making it more difficult to detect malicious encrypted traffic, which affects the performance of baseline methods.

F. Ablation Study

In order to fully verify the effect of each component of TGE-ETD, we conducted ablation experiments on each component separately. The results of the ablation experiment are shown in Table V. We refer to the design principles to analyze the results of the ablation experiment one by one and obtain the following results.

1) The Contribution of the Header and Payload Parts of the Traffic Packet: Compared to the raw byte features of both the header and the payload, we only use the raw bytes of the header (or payload) as input, marked w/o-payload (or w/o-header). When only the header part is used for detection, the model performance drops by about 1%-2%. When only the payload part is used for detection, the model performance drops by about 5%-30%. It shows that the header and payload parts of the traffic packet contribute to the overall encrypted traffic detection. When both are used as input at the same time, the model achieves the best effect. Then, through comparison, we found that the header part contributes more to encrypted traffic detection than the payload part. It is because the header part is plain text and carries the communication characteristics of the traffic. In addition, it is difficult to extract

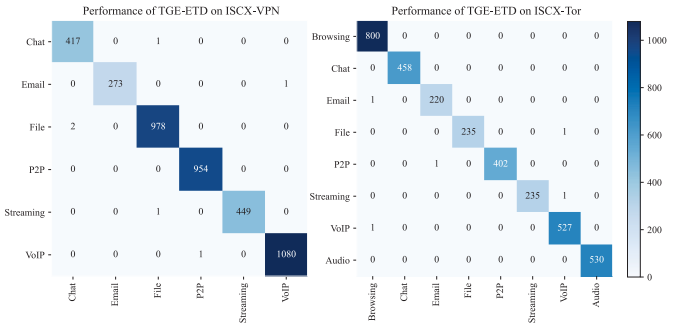


Fig. 7. Performances of TGE-ETD on the ISCX-VPN and ISCX-Tor.

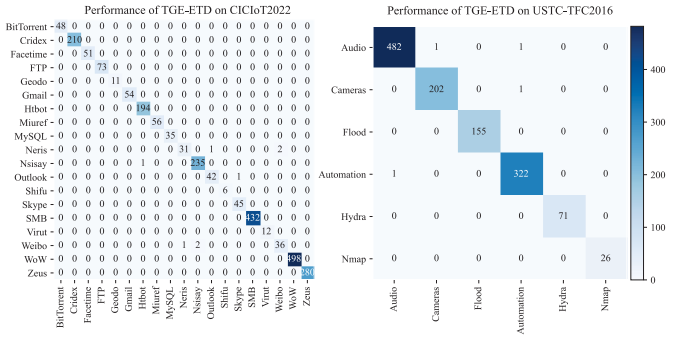


Fig. 8. Performances of TGE-ETD on USTC-TFC2016 and CICIoT2022.

features because the payload part is ciphertext. It can be also observed that the contribution of the payload part is higher in the malicious encrypted traffic dataset. This is because the payload part of the data packets sent by different attacks is considerably different, while the data packets of the same type of attack have similar load information, which makes the payload features easier to distinguish. The encrypted traffic data packets in VPN and Tor environments are obfuscated, making the payload part more difficult to distinguish.

2) The Contribution of the Raw Byte Modeling Method: Compared to the co-occurrence graph and the word frequency graph to model the raw bytes, we only use the co-occurrence graph (or the word frequency graph) to model the encrypted traffic packet, marked as w/o-fg (or w/o-oc). The effect of the model is poor (down 70%) in the absence of word frequency graph modeling. The frequency features of bytes and the original byte feature embedded as node features are missing. The original byte graph that only retains the co-occurrence relationship cannot effectively represent the encrypted traffic packet. In the absence of co-occurrence graph modeling, the model effect drops by about 10%. It should be noted that the model ACC drops significantly (about 50%) in the Tor dataset, which shows that the byte co-occurrence relationship is an important feature of the encrypted traffic classification method based on raw bytes.

3) Effectiveness Evaluation of the Feature Fusion Method Based on Global Attention Pooling: Compared to using global self-attention to fuse node features to obtain the representation of the graph, and then concatenating the features of the two parts to obtain the final representation using linear layer transformation (marked TGE-ETD-att), we only process node features by averaging sampling to obtain the representation, marked as TGE-ETD-mean. By comparing the results of the average pooling feature fusion and the global self-attention feature fusion methods, the difference between the two is approximately 0.5%, which verifies the effectiveness of the global self-attention feature fusion method.

G. Stability Analysis of TGE-ETD

The results of the stability analysis are shown in Fig. 7 and Fig. 8. We plot the heat maps based on the confusion matrix. The following conclusions can be drawn based on the results shown in the figure. First, TGE-ETD achieved high accuracy in classifying samples of each category in the four

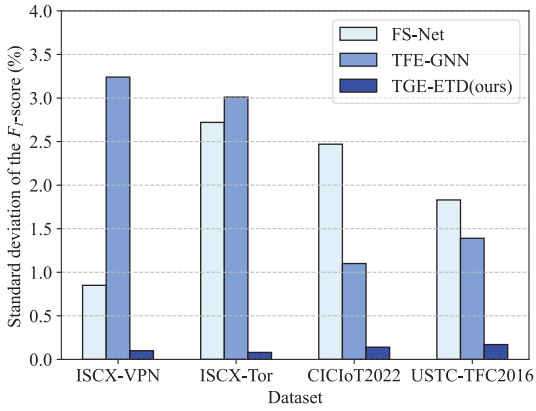


Fig. 9. Standard deviation of the F_1-score.

datasets, and there is no uneven classification effect of samples of different categories. Second, TGE-ETD is less affected by the number of categories. TGE-ETD has high accuracy on different numbers of categories including 6, 8 and 19. Finally, TGE-ETD can achieve high accuracy even when the ratio of the number of samples in the two malicious traffic detection datasets reaches 1:20, which is unbalanced.

In order to verify the stability of the model, we repeated each experiment ten times and took the average of the ten experimental results as the final result. Fig. 9 shows the comparison of the standard deviation of F_1 of different methods. For ET-BERT and YaTC, we only repeated the fine-turning part ten times. The ten experimental results of the fine-tuning stage of the pre-trained model are the same, so the standard deviation is 0. From the results in the figure, the standard deviation of TGE-ETD is smaller than those of FS-Net and TFE-GNN, which means that TGE-ETD is more stable.

H. GNN Architecture Variants Study

To verify the effectiveness of the GraphSAGE module in extracting traffic graph features, we use classic GNN models including GCN [62] and GIN [63] to replace the GraphSAGE module as variants for comparison.

As shown in Fig. 10, the results show that the encrypted traffic detection method based on GraphSAGE performs better than other variants, especially in malicious encrypted traffic detection tasks. This is because GraphSAGE learns the embedded expression of the target node itself by aggregating

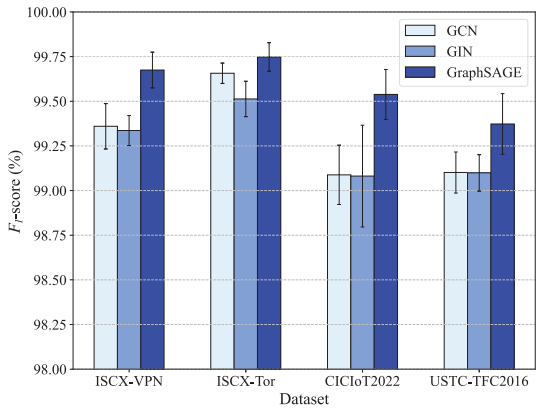


Fig. 10. Comparison results of GNN models. The figure shows the F_1-score results of different GNN models in the twin graph encoder stage.

TABLE VI
PERFORMANCE COMPARISON ON UAV-NIDD (%)

Model	ACC	PR	F1	RC
FNN	97.18	95.76	95.20	97.18
MAML	75.71	69.54	71.09	72.70
TFE-GNN	99.86±0.04	99.89±0.03	93.01±2.14	88.67±3.05
YaTC	99.95±0.03	99.97±0.02	99.96±0.77	99.23±1.46
TGE-ETD	**99.96±0.05**	**99.98±0.03**	**99.97±0.04**	**99.97±0.04**

TABLE VII
THE COMPARISON OF THE MODEL FLOPS AND PARAMETERS

Model	FLOPS(M)	Parameters(M)
FS-NET	1.0e+02	3.2e+00
ET-BERT	1.1e+04	8.6e+01
YaTC	2.1e+00	2.3e+00
GraphDAPP	3.8e-02	1.1e-02
TFE-GNN	2.2e+03	4.4e+01
TGE-ETD	8.5e+01	8.4e-02

TABLE VIII
CONSUMPTION ANALYSIS OF TGE-ETD ON UAV DEVICES

Device	Communication Module	Max Download Speed	Overhead	Latency
DJI-Air 3S [65]	Wi-Fi 5	30 MB/s	1.13%	11.33 ms
DJI Mavic 3 Pro [65]	Wi-Fi 6	80 MB/s	0.43%	4.25 ms
DJI Flip [65]	Wi-Fi 5	30 MB/s	1.13%	11.33 ms
ANAFI Ai [66]	Wi-Fi 4 & 4 G	10 MB/s	3.40%	34 ms
ANAFI USA [66]	Wi-Fi 4	20 MB/s	1.70%	17 ms
Yuneec H520 [67]	Wi-Fi 5	30 MB/s	1.13%	11.33 ms

the feature information of the node neighbors and has better generalization ability than GCN and GIN. At the same time, all three graph neural networks can achieve a high F_1, which reflects the effectiveness of the traffic byte graph modeling.

I. Effectiveness of TGE-ETD on UAV Traffic Dataset

The UAV-NIDD [64] dataset contains encrypted traffic collected from real-world UAV networks. To validate the effectiveness of TGE-ETD in UAV network traffic detection, we conducted comparative experiments on the UAV-NIDD dataset. FNN [64] and MAML [64], which are deep learning–based approaches for UAV network traffic classification, were included as baseline methods for comparison. Table VI reports the mean and standard deviation of ten experiments. The experimental results demonstrate that TGE-ETD achieves strong performance in UAV network traffic detection, maintaining high classification accuracy. Compared with existing UAV traffic classification approaches, TGE-ETD improves accuracy by approximately 2%–20%. Compared to baseline methods, TGE-ETD can effectively capture discriminative features of encrypted traffic.

J. Model Complexity Analysis

1) Analysis of Model Parameters Count and Floating Point Operations: To assess the balance between model performance and complexity, we provide floating point operations (FLOPs) and parameters of different methods, as shown in Table VII. Combining the results in Table VII with those in Table IV, we can conclude that TGE-ETD achieves improvement at low model complexity. Compared with traditional deep learning and pretraining methods, TGE-ETD has the smallest number of parameters and the second lowest number of FLOPs. Although GraphDAPP has the lowest complexity, TGE-ETD improves performance by 75% compared to GraphDAPP. Although YaTC achieves comparable results on the ISCX-VPN and ISCX-Tor datasets, the number of parameters of YaTC is about 27 times that of TGE-ETD. In addition, the pre-training phase of YaTC is time-consuming due to the large amount of additional data and high model complexity during pre-training. Compared with the advanced GNN-based method TFE-GNN, which reduces the number of FLOPs by 25 times and the number of parameters by 500 times, TGE-ETD not only reduces complexity, but also achieves improvement.

2) Overhead Analysis on UAV Devices: To verify the feasibility of deploying the model in practice, we collected the hardware information of six UAV devices from three different UAV manufacturers. As shown in Table VIII, most UAV devices are equipped with Wi-Fi modules for communication, with maximum download speeds ranging from 10 to 80 MB/s (Mega Byte per second). We saved the TGE-ETD model parameter file, which is 0.34 MB (Mega Byte). Under these conditions, the latency incurred during model updating ranges from 4.25 to 34 ms (millisecond), achieving millisecond-level performance. The bandwidth occupancy caused by model updates accounts for approximately 0.43% to 3.40% of the available per-second bandwidth on the UAV side.

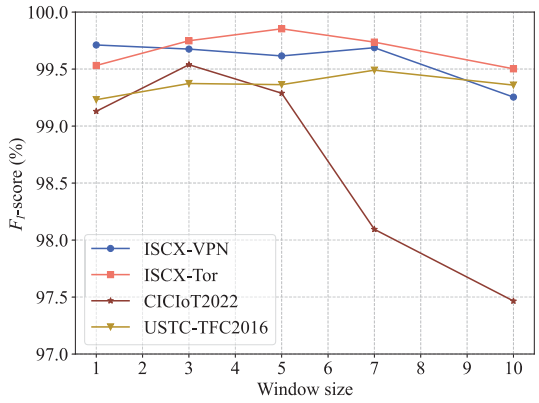


Fig. 11. Model sensitivity analysis. Different window size of the co-occurrence graph construction method on four datasets.

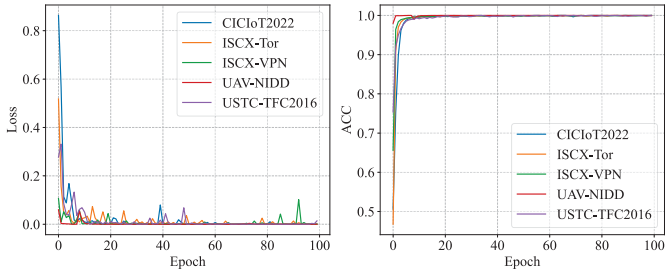


Fig. 12. ACC and loss convergence curves of TGE-ETD.

TABLE IX
TRAINING TIME AND RESOURCE CONSUMPTION OF TGE-ETD

Dataset	Training Time	Conver. Epoch	Conver. Time	GPU Consumption	Energy Consumption
ISCX-VPN	5044.2 s	20	1008.8 s	1.8 G	128 W
ISCX-Tor	4215.2 s	20	843.0 s	1.6 G	128 W
CICIoT2022	1419.4 s	20	283.9 s	1.6 G	114 W
USTC-TFC2016	3173.6 s	20	634.7 s	1.7 G	116 W
UAV-NIDD	606.1 s	20	121.2 s	1.4 G	112 W

K. Model Sensitivity Analysis

To investigate the impact of sliding window size, we conducted a sensitivity analysis experiment. As shown in Fig. 11, the model performs relatively well when the sliding window size for constructing the co-occurrence graph is small. As the window size increases, there are more edges in the co-occurrence graph, and the relationship between different bytes becomes less distinguishable, making it difficult for the model to learn the relationship features between the original bytes.

L. Training Time and Energy Consumption Analysis

We evaluated the training efficiency of TGE-ETD by measuring its training time, convergence (recorded as Conver.) speed, and energy consumption, with the results summarized in Table IX. For UAV network traffic data, TGE-ETD requires only 606.1 s (second) to complete the training. The ACC and Loss curves of TGE-ETD are illustrated in Fig. 12, from which it can be observed that the model converges within 20 epochs. In addition, we recorded GPU memory consumption and power usage during the training process. TGE-ETD requires less than 2 GB (Giga Byte) of GPU memory and consumes 130 W (Watt) of GPU power.

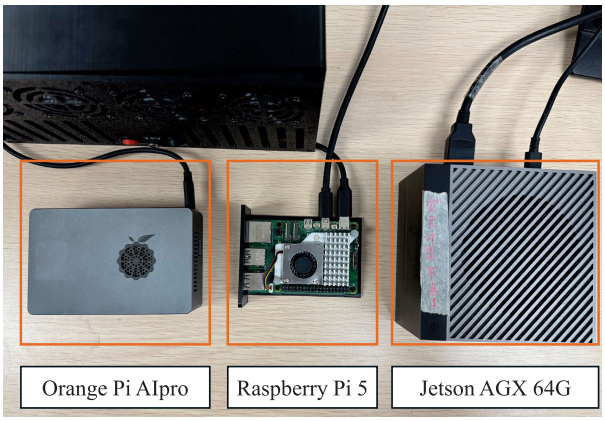


Fig. 13. Experimental environment on realistic UAV hardware.

TABLE X
TESTING RESULTS ON REALISTIC UAV HARDWARE

Device	Memory Usage	Inference Latency	Latency per Packet
Raspberry Pi 5	11376.05 MB	10.0134 s	7.9 ms
Orange Pi AIpro	11233.82 MB	16.9861 s	13.4 ms
Jetson AGX 64G	11419.93 MB	12.3078 s	9.7 ms
Desktop	11196.35 MB	0.5322 s	0.4 ms

M. Testing on Realistic UAV Hardware

We conducted inference experiments on three devices: Raspberry Pi 5 [68], Orange Pi AIpro [69], and Jetson AGX 64 G [70], as shown in Fig. 13. The test dataset consists of 1267 packets from the UAV-NIDD dataset. Table X reports the memory footprint and inference latency on each platform. The results show that our model achieves millisecond-level per-packet inference latency on ARM-based devices, demonstrating the feasibility of deployment in UAV environments.

N. Attention Visualization of Encrypted Traffic of UAV

To identify bytes that have a greater impact on model classification in UAV traffic, we visualized the attention weights of the nodes in the constructed traffic graphs. We visualized normal traffic in the UAV-NIDD dataset, and the results are shown in Fig. 14. For Normal traffic in the UAV network, most nodes exhibit attention weights close to zero, while higher attention values are mainly concentrated in the byte index ranges of 79–88 and 240–255. The experimental results reveal a long-tail distribution of attention weights, indicating that only a small number of nodes play a dominant role in the classification decision. It is shown that TGE-ETD can effectively capture discriminative structural features within encrypted traffic.

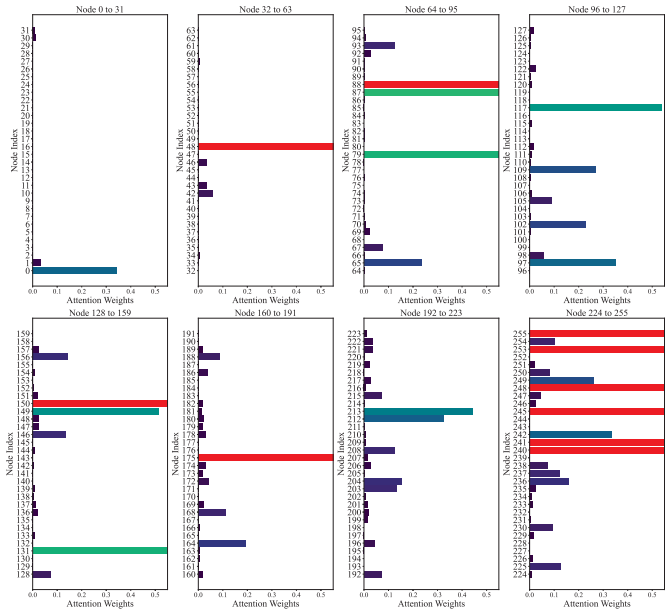


Fig. 14. Attention weights of normal traffic in UAV-NIDD. The nodes with higher attention are concentrated in the ranges of 79-88 and 240-255.

VI. Future Works and Conclusion

A. Future Works

1) Defense Against Network Attacks in Heterogeneous Large-Scale UAV Networks: With the rapid advancement of the low-altitude economy, UAV networks are inevitably evolving toward large-scale and heterogeneous deployments. This trend introduces increasingly complex network attack surfaces and poses significant challenges for the performance and scalability of existing defense mechanisms. To address these issues, future research can explore advanced and proactive defense strategies such as decentralized federated learning and moving target defense to enable comprehensive protection.

2) Accurate Encrypted Traffic Modeling Under Massive UAV Network Traffic: As UAV networks continue to grow in scale, the volume of encrypted traffic will increase, significantly complicating accurate traffic detection. Therefore, there is a pressing need to develop more precise and efficient modeling techniques for encrypted traffic. The graph-based modeling approach proposed in this work currently relies solely on raw byte-level information. Future efforts should focus on improving this modeling framework by integrating dynamic features such as network connectivity patterns to improve detection performance.

3) Detection of Unknown Encrypted Traffic Attacks in Open UAV Networks: Most existing malicious encrypted traffic detection methods, including the one proposed in this work, rely heavily on large amounts of labeled data for training. However, manual labeling is costly and time consuming, and these methods struggle to identify previously unseen attacks. To overcome these limitations, future work will explore few-shot learning and unsupervised detection techniques to develop open-world detection frameworks capable of identifying unknown encrypted traffic attacks in UAV networks.

4) Timely Encrypted Traffic Detection in Dynamic UAV Networks: UAV networks exhibit highly dynamic and adaptive characteristics, and users often have stringent requirements regarding the timeliness of the collected information. Consequently, the traffic data within UAV networks possess temporal properties, such as the age of information. In future work, we plan to incorporate the temporal characteristics of UAV network traffic and design an efficient malicious encrypted traffic detection method tailored for dynamic UAV networks.

5) Adversarial Robustness in Practical UAV Networks: In practical UAV networks, attackers can manipulate traffic to conceal malicious activities. At the traffic content level, there exist potential adversarial risks, including adjusting padding fields, fragmentation boundaries, or other redundant structures within encrypted payloads, which may alter byte-level statistical characteristics without violating encryption integrity. At the network level, attackers can further evade detection mechanisms by manipulating packet lengths, transmission timing, or traffic burst patterns. Therefore, in our future work we will investigate encrypted traffic detection in adversarial UAV environments. We will explore the use of advanced learning paradigms, including large language models, to enhance robustness against adaptive evasion strategies.

B. Conclusion

In this paper, we have proposed a novel resilient UAV network architecture based on malicious encrypted traffic detection. By decoupling the training and inference processes of the encrypted traffic detection model, the proposed architecture addresses the computational limitations of UAV devices. In addition, we have introduced a new end-to-end encrypted traffic detection model, termed TGE-ETD. To effectively extract features from encrypted traffic, we have proposed a graph-based byte-level traffic modeling method. Specifically, two types of byte graphs, namely the frequency graph and the co-occurrence graph, are constructed to represent encrypted traffic. A twin graph encoder is then designed to transform the byte graphs into feature vectors. Subsequently, a global attention pooling is employed to obtain the final feature representation of encrypted traffic. Extensive experiments conducted on four public datasets and one real-world UAV dataset demonstrate the effectiveness and superiority of TGE-ETD.

References

[1] M. K. C. Shisher, A. Piaseczny, Y. Sun, and C. G. Brinton, "Computation and communication co-scheduling for timely multi-task inference at the wireless edge," in *Proc. IEEE Conf. Comput. Commun.*, May 2025, pp. 1–10.

[2] C. Zhao et al., "Foes or friends: Embracing ground effect for edge detection on lightweight drones," in *Proc. 30th Annu. Int. Conf. Mobile Comput. Netw.*, Dec. 2024, pp. 1377–1392.

[3] B.-B. Zhang, D. Zhang, R. Song, B. Wang, Y. Hu, and Y. Chen, "RF-search: Searching unconscious victim in smoke scenes with RF-enabled drone," in *Proc. 29th Annu. Int. Conf. Mobile Comput. Netw.*, Oct. 2023, pp. 1–15.

[4] E. Sie, Z. Liu, and D. Vasisht, "BatMobility: Towards flying without seeing for autonomous drones," in *Proc. 29th Annu. Int. Conf. Mobile Comput. Netw.*, Oct. 2023, pp. 1–16.

[5] C. Lei, W. Feng, P. Wei, Y. Chen, N. Ge, and S. Mao, "Edge information hub: Orchestrating satellites, UAVs, MEC, sensing and communications for 6G closed-loop controls," *IEEE J. Sel. Areas Commun.*, vol. 43, no. 1, pp. 5–20, Jan. 2025.

[6] H. Sun et al., "All-sky autonomous computing in UAV swarm," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 13258–13274, Dec. 2024.

[7] X. Wang et al., "A survey on security of UAV swarm networks: Attacks and countermeasures," *ACM Comput. Surv.*, vol. 57, no. 3, pp. 1–37, Mar. 2025.

[8] M. Mozaffari, W. Saad, M. Bennis, Y.-H. Nam, and M. Debbah, "A tutorial on UAVs for wireless networks: Applications, challenges, and open problems," *IEEE Commun. Surveys Tut.*, vol. 21, no. 3, pp. 2334–2360, 3rd Quart., 2019.

[9] O. Ceviz, S. Sen, and P. Sadioglu, "A survey of security in UAVs and FANETs: Issues, threats, analysis of attacks, and solutions," *IEEE Commun. Surveys Tut.*, vol. 27, no. 5, pp. 3227–3265, Oct. 2025.

[10] L. Cai et al., "Secure physical layer communications for low-altitude economy networking: A survey," *IEEE Commun. Surveys Tut.*, vol. 28, pp. 2497–2530, 2026.

[11] S. Tian et al., "Reasoning techniques meet GraphRAG: Advancing LLM for wireless network cyber defense," *IEEE Wireless Commun.*, early access, Feb. 27, 2026, doi: 10.1109/MWC.2026.3659831.

[12] X. Li et al., "Protect NTN-IoT security by malicious traffic detection: A multidimensional hypergraph learning approach," *IEEE Internet Things J.*, vol. 13, no. 3, pp. 3594–3607, Feb. 2026.

[13] J. Sharma and P. S. Mehra, "Secure communication in IoT-based UAV networks: A systematic survey," *Internet Things*, vol. 23, Oct. 2023, Art. no. 100883.

[14] W. Zhai, L. Liu, Y. Ding, S. Sun, and Y. Gu, "ETD: An efficient time delay attack detection framework for UAV networks," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 2913–2928, 2023.

[15] H. Sedjelmaci, S. M. Senouci, and N. Ansari, "A hierarchical detection and response system to enhance security against lethal cyber-attacks in UAV networks," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 48, no. 9, pp. 1594–1606, Sep. 2018.

[16] H. Sedjelmaci, S. M. Senouci, and M.-A. Messous, "How to detect cyber-attacks in unmanned aerial vehicles network?," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2016, pp. 1–6.

[17] Q. Zeng and F. Nait-Abdesselam, "Enhancing UAV network security: A human-in-the-loop and GAN-based approach to intrusion detection," *IEEE Internet Things J.*, vol. 12, no. 12, pp. 20870–20884, Jun. 2025.

[18] T. Li et al., "AeroGuard: Towards real-time UAV fault detection with hybrid models," *IEEE Trans. Mobile Comput.*, vol. 25, no. 6, pp. 9075–9088, Jun. 2026.

[19] Z. Wang, K. W. Fok, and V. L. L. Thing, "Machine learning for encrypted malicious traffic detection: Approaches, datasets and comparative study," *Comput. Secur.*, vol. 113, Feb. 2022, Art. no. 102542.

[20] G. Aceto, A. Dainotti, W. de Donato, and A. Pescape, "PortLoad: Taking the best of two worlds in traffic classification," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM)*, San Diego, CA, USA, Mar. 2010, pp. 1–5.

[21] C. Xu, S. Chen, J. Su, S. M. Yiu, and L. C. K. Hui, "A survey on regular expression matching for deep packet inspection: Applications, algorithms, and hardware platforms," *IEEE Commun. Surveys Tut.*, vol. 18, no. 4, pp. 2991–3029, 4th Quart., 2016.

[22] A. S. Shekhawat, F. D. Troia, and M. Stamp, "Feature analysis of encrypted malicious traffic," *Expert Syst. Appl.*, vol. 125, pp. 130–141, Jul. 2019.

[23] A. Este, F. Gringoli, and L. Salgarelli, "Support vector machines for TCP traffic classification," *Comput. Netw.*, vol. 53, no. 14, pp. 2476–2490, Sep. 2009.

[24] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," in *Proc. IEEE Int. Conf. Intell. Secur. Informat. (ISI)*, Jul. 2017, pp. 43–48.

[25] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE Conf. Comput. Commun.*, Apr. 2019, pp. 1171–1179.

[26] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, Apr. 2022, pp. 633–642.

[27] R. Zhao et al., "Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation," in *Proc. AAAI Conf. Artif. Intell.*, 2023, vol. 37, no. 4, pp. 5420–5427.

[28] H. Y. He, Z. Guo Yang, and X. N. Chen, "PERT: Payload encoding representation from transformer for encrypted traffic classification," in *Proc. ITU Kaleidoscope: Ind.-Driven Digit. Transformation (ITU K)*, Dec. 2020, pp. 1–8.

[29] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2367–2380, 2021.

[30] H. Zhang et al., "TFE-GNN: A temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification," in *Proc. ACM Web Conf.*, Apr. 2023, pp. 2066–2075.

[31] D. Jiang et al., "Toward synthetic network traffic generating in NTN-enabled IoT: A generative AI approach," *IEEE Internet Things J.*, vol. 12, no. 2, pp. 2174–2187, Jan. 2025.

[32] Y. Wang, C. Wang, J. Zhan, W. Ma, and Y. Jiang, "Text FCG: Fusing contextual information via graph learning for text classification," *Expert Syst. Appl.*, vol. 219, Jun. 2023, Art. no. 119658.

[33] B. Zhao, S. Guan, M. Chen, J. Wu, C. Qiao, and Y. Xiao, "Efficient and accurate privacy-preserving task allocation for unmanned aerial vehicle-based mobile crowdsensing," *IEEE Trans. Netw. Sci. Eng.*, vol. 13, pp. 5638–5653, 2026.

[34] N. Kumar and A. Chaudhary, "Secure and lightweight mechanism for UAV-GCS and UAV-UAV authentication based on PUF," *IEEE Internet Things J.*, early access, Feb. 27, 2026, doi: 10.1109/JIOT.2026.3669029.

[35] K.-D. Lu, B.-X. Zhang, Y. Xu, Y. Shen, and Z.-G. Wu, "MoARNN-AM: Multi-objective automated recurrent neural network with attention mechanism for cyber-attack detection of UAV," *IEEE Trans. Consum. Electron.*, vol. 72, no. 1, pp. 1738–1749, Feb. 2026.

[36] L. Xie, T. Xiao, M. Zhou, W. Nie, and X. Yang, "Robust two-layer UAV spoofing detection based on multi-source/multi-dimensional data analysis for low-altitude networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 13, pp. 6737–6753, 2026.

[37] Y. Xiao et al., "RF-based identification framework against unauthorized UAV networking in low-altitude economy," *IEEE Trans. Netw. Sci. Eng.*, vol. 13, pp. 6538–6555, 2026.

[38] Y. Qi, L. Xu, B. Yang, Y. Xue, and J. Li, "Packet classification algorithms: From theory to practice," in *Proc. IEEE INFOCOM*, Apr. 2009, pp. 648–656.

[39] P. M. Fivos Constantinou, "Identifying known and unknown peer-to-peer traffic," in *Proc. 5th IEEE Int. Symp. Netw. Comput. Appl. (NCA)*, Jul. 2006, pp. 93–102.

[40] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "AppScanner: Automatic fingerprinting of smartphone apps from encrypted network traffic," in *Proc. IEEE Eur. Symp. Secur. Privacy (EuroS&P)*, Mar. 2016, pp. 439–454.

[41] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "Robust smartphone app identification via encrypted network traffic analysis," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 1, pp. 63–78, Jan. 2018.

[42] C. Liu, Z. Cao, G. Xiong, G. Gou, S.-M. Yiu, and L. He, "MaMPF: Encrypted traffic classification based on multi-attribute Markov probability fingerprints," in *Proc. IEEE/ACM 26th Int. Symp. Quality Service (IWQoS)*, Jun. 2018, pp. 1–10.

[43] M. Shen, Y. Liu, L. Zhu, K. Xu, X. Du, and N. Guizani, "Optimizing feature selection for efficient encrypted traffic classification: A systematic approach," *IEEE Netw.*, vol. 34, no. 4, pp. 20–27, Jul. 2020.

[44] M. Lotfollahi, M. Jafari Siavoshani, R. Shirali Hossein Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Comput.*, vol. 24, no. 3, pp. 1999–2012, Feb. 2020.

[45] X. Wang, S. Chen, and J. Su, "App-net: A hybrid neural network for encrypted mobile traffic classification," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, Jul. 2020, pp. 424–429.

[46] Q. Meng et al., "Detection of unknown attacks through encrypted traffic: A Gaussian prototype-aided variational autoencoder framework," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 10652–10667, 2025.

[47] S. Zhang, F. Yu, H. Zhou, and B. Li, "MalDetectFormer: Leveraging sparse SpatioTemporal information for effective malicious traffic detection," in *Proc. AAAI Conf. Artif. Intell.*, 2025, vol. 39, no. 21, pp. 22533–22541.

[48] M. Shen, J. Wu, K. Ye, K. Xu, G. Xiong, and L. Zhu, "Robust detection of malicious encrypted traffic via contrastive learning," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 4228–4242, 2025.

[49] Y. Ding, G. Zhu, D. Chen, X. Qin, M. Cao, and Z. Qin, "Adversarial sample attack and defense method for encrypted traffic data," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 10, pp. 18024–18039, Oct. 2022.

[50] D. Xu et al., "LMT-SDNN: A lightweight malicious traffic detection method for the Internet of Things based on multiteacher distillation," *IEEE Internet Things J.*, vol. 13, no. 8, pp. 15665–15677, Apr. 2026.

[51] S. Cai, Y. Zhao, W. Zhao, J. Chen, S. Wang, and B. Gu, "MD-CGM: Malicious traffic detection model based on CycleGAN and multi-head self-attetion mechanism," *Future Gener. Comput. Syst.*, vol. 181, Aug. 2026, Art. no. 108421.
[52] X. Han et al., "DE-GNN: Dual embedding with graph neural network for fine-grained encrypted traffic classification," *Comput. Netw.*, vol. 245, May 2024, Art. no. 110372.
[53] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. North Amer. Chapter Assoc. Comput. Linguistics*, 2019, pp. 4171–4186.
[54] L. Yang et al., "Robustness matters: Pre-training can enhance the performance of encrypted traffic analysis," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 10588–10603, 2025.
[55] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1024–1034.
[56] W. Cong, M. Ramezani, and M. Mahdavi, "On provable benefits of depth in training graph convolutional networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 9936–9949.
[57] Y. Li, R. Zemel, M. Brockschmidt, and D. Tarlow, "Gated graph sequence neural networks," in *Proc. Int. Conf. Learn. Represent.*, 2016, pp. 1–20.
[58] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Jan. 2017, pp. 712–717.
[59] S. Dadkhah, H. Mahdikhani, P. K. Danso, A. Zohourian, K. A. Truong, and A. A. Ghorbani, "Towards the development of a realistic multidimensional IoT profiling dataset," in *Proc. 19th Annu. Int. Conf. Privacy, Secur. Trust (PST)*, Aug. 2022, pp. 1–11.
[60] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related features," in *Proc. 2nd Int. Conf. Inf. Syst. Secur. Privacy*, 2016, pp. 407–414.
[61] A. Habibi Lashkari, G. Draper Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of tor traffic using time based features," in *Proc. 3rd Int. Conf. Inf. Syst. Secur. Privacy*, 2017, pp. 253–262.
[62] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Represent.*, 2017, pp. 1–14.
[63] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?," in *Proc. Int. Conf. Learn. Represent.*, 2018, pp. 1–17.
[64] H. J. Hadi, Y. Cao, M. K. Khan, N. Ahmad, Y. Hu, and C. Fu, "UAV-NIDD: A dynamic dataset for cybersecurity and intrusion detection in UAV networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 4, pp. 2739–2757, Jul. 2025.
[65] DJI.(2026). *Consumer Drones Comparison*. [Online]. Available: https://www.dji.com/products/comparison-consumer-drones
[66] Parrot.(2026). *Professional Drones Built for Work*. [Online]. Available: https://www.parrot.com/en/drones
[67] Yuneec.(2026). *Professional Grade Unmanned Aerial Vehicle*. [Online]. Available: https://yuneec.online/h520-series/
[68] RaspberryPi.(2026). *Raspberry PI 5*. [Online]. Available: https://www.raspberrypi.com/products/raspberry-pi-5/
[69] OrangePi.(2026). *OrangePi Aipro (20T) Built for AI*. [Online]. Available: http://www.orangepi.org/html/hardWare/computerAndMicrocontrollers/details/Orange-Pi-AIpro(20t).html
[70] NVIDIA.(2026). *NVIDIA Jetson Orin: Next-Level AI Performance for Next-Gen Robotics and Edge Solutions*. [Online]. Available: https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/

Xuzeng Li (Graduate Student Member, IEEE) received the B.Eng. degree from Beijing Jiaotong University, Beijing, China, in 2020, where he is currently pursuing the Ph.D. degree. His research interests include UAV networks security, traffic detection, and federated learning.

Tao Zhang (Member, IEEE) received the joint B.S. degree in Internet of Things engineering from Beijing University of Posts and Telecommunications (BUPT) and the Queen Mary University of London in 2018 and the Ph.D. degree in computer science and technology from BUPT in 2023. He is currently an Associate Professor and an Advisor of Ph.D. Candidates with the School of Cyberspace Science and Technology, Beijing Jiaotong University. His publications include ESI highly cited papers and well-archived international journals and proceedings, such as IEEE COMMUNICATIONS SURVEYS AND TUTORIALS, IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS, IEEE TRANSACTIONS ON COMMUNICATIONS, IEEE TRANSACTIONS ON COGNITIVE COMMUNICATIONS AND NETWORKING, IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS, IEEE TRANSACTIONS ON NETWORK SCIENCE AND ENGINEERING, and IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY. His research interests include the IoV security, moving target defense, and federated learning. He was a recipient of the Best Paper Award from NaNA 2018, IWCMC 2021, DIONE 2024, ICA3PP 2024, and KSEM 2025; and the Outstanding Paper Award from iThings 2023, SmartCity 2024, and Trustcom 2025. His Ph.D. thesis was awarded the Outstanding Doctoral Dissertation by China Institute of Communications and BUPT. He is the TPC chair and a PC member for some international conferences and workshops. He serves as an Associate Editor for IEEE NETWORKING LETTERS, IEEE DATA DESCRIPTIONS, *Computer Networks*, AD HOC NETWORKS, and *EURASIP Journal on Information Security*; and a Guest Editor for *Remote Sensing*, *Electronics*, and *Chinese Journal of Network and Information Security*.

Jiacheng Wang (Member, IEEE) received the Ph.D. degree from the School of Communications and Information Engineering, Chongqing University of Posts and Telecommunications, Chongqing, China. He is currently a Research Fellow with the College of Computing and Data Science, Nanyang Technological University, Singapore. His research interests include wireless sensing security, generative AI, semantic communications, and low-altitude wireless networks.

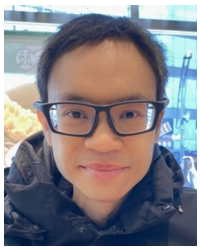
Jiangtian Nie received the B.Eng. degree in electronics and information engineering from the Huazhong University of Science and Technology, Wuhan, China, in 2016, and the Ph.D. degree from ERI@N, Interdisciplinary Graduate School, Nanyang Technological University (NTU), Singapore, in 2021. She was a Visiting Student with Princeton University and the University of Waterloo. She is currently a Lecturer with the Computer Science Department, University of Aberdeen, U.K. Her research interests include edge intelligence, wireless communications, and the Internet of Things.

Jian Wang received the Ph.D. degree in cryptography from Beijing University of Posts and Telecommunications in 2008. He is currently a Professor with the School of Cyberspace Science and Technology, Beijing Jiaotong University, China. His current research interests include big data security and analysis, cryptography application and authentication technology, and computer forensics technology.

Xuangou Wu (Member, IEEE) received the Ph.D. degree in computer software and theory from the University of Science and Technology of China (USTC), Hefei, China, in 2013. From September 2018 to September 2019, he was a Visiting Scholar with the Department of Computer Science and Engineering, The Hong Kong University of Science and Technology (HKUST). He is currently a Professor with the School of Computer Science and Technology, Anhui University of Technology, Ma'anshan, China. His research interests include the Internet of Things, network security, federated learning, and privacy protection.

Zhen Han received the Ph.D. degree from China Academy of Engineering Physics, Beijing, China, in 1991. He is currently a Professor with the School of Cyberspace Science and Technology, Beijing Jiaotong University, Beijing. He has authored or co-authored more than 100 papers in various journals and international conferences. His main research interests include information security architecture and trusted computing.

Jiqiang Liu (Senior Member, IEEE) received the B.S. and Ph.D. degrees from Beijing Normal University, Beijing, China, in 1994 and 1999, respectively. He is currently a Professor with the School of Cyberspace Science and Technology, Beijing Jiaotong University, Beijing. He has published over 200 scientific papers in various journals and international conferences. His main research interests include privacy-preserving, the IoT security, and cryptographic protocols.



Dusit Niyato (Fellow, IEEE) received the B.Eng. degree from the King Mongkut's Institute of Technology Ladkrabang, Thailand, in 1999, and the Ph.D. degree in electrical and computer engineering from the University of Manitoba, Canada, in 2008. He is currently a Professor with the School of Computer Science and Engineering, Nanyang Technological University, Singapore. His research interests include sustainability, edge intelligence, decentralized machine learning, and incentive mechanism design.

Dong In Kim (Life Fellow, IEEE) received the Ph.D. degree in electrical engineering from the University of Southern California, Los Angeles, CA, USA, in 1990. He was a Tenured Professor with the School of Engineering Science, Simon Fraser University, Burnaby, BC, Canada. He is currently a Distinguished Professor with the College of Information and Communication Engineering, Sungkyunkwan University, Suwon, South Korea. He is a fellow of Korean Academy of Science and Technology and a Life Member of the National Academy of Engineering of Korea. He was a first recipient of the NRF of Korea Engineering Research Center in Wireless Communications for RF Energy Harvesting from 2014 to 2021. He received several research awards, including the 2023 IEEE ComSoc Best Survey Paper Award and the 2022 IEEE Best Land Transportation Paper Award. He was selected as the 2019 recipient of the IEEE ComSoc Joseph LoCicero Award for Exemplary Service to Publications. He has been listed as a 2020/2022 Highly Cited Researcher by Clarivate Analytics. He was the General Chair of IEEE ICC 2022, Seoul. From 2001 to 2024, he served as an Editor, the Editor-at-Large, and an Area Editor of Wireless Communications—I for IEEE TRANSACTIONS ON COMMUNICATIONS. From 2002 to 2011, he served as an Editor and a Founding Area Editor of Cross-Layer Design and Optimization for IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS. From 2008 to 2011, he served as the Co-Editor-in-Chief for IEEE/KICS JOURNAL OF COMMUNICATIONS AND NETWORKS. He served as the Founding Editor-in-Chief for IEEE WIRELESS COMMUNICATIONS LETTERS from 2012 to 2015.